JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AtomicBoolean

java.lang.Object
    java.util.concurrent.atomic.AtomicBoolean

All Implemented Interfaces:

Serializable

```
public class AtomicBoolean
extends Object
implements Serializable
```

A `boolean` value that may be updated atomically. See the `VarHandle` specification for descriptions of the properties of atomic accesses. An `AtomicBoolean` is used in applications such as atomically updated flags, and cannot be used as a replacement for a `Boolean`.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`AtomicBoolean()`	Creates a new `AtomicBoolean` with initial value `false`.
`AtomicBoolean (boolean initialValue)`	Creates a new `AtomicBoolean` with the given initial value.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`final boolean`	`compareAndExchange (boolean expectedValue, boolean newValue)`	Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as

		specified by VarHandle.compareAndExchange(Object...).
final boolean	compareAndExchangeAcquir (boolean expectedValue, boolean newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeAcquire(Obje
final boolean	compareAndExchangeReleas (boolean expectedValue, boolean newValue)	Atomically sets the value to newValue if the current value, referred to as the *witness value*, == expectedValue, with memory effects as specified by VarHandle.compareAndExchangeRelease(Obje
final boolean	compareAndSet (boolean expectedValue, boolean newValue)	Atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.compareAndSet(Object...).
final boolean	get()	Returns the current value, with memory effects as specified by VarHandle.getVolatile(Object...).
final boolean	getAcquire()	Returns the current value, with memory effects as specified by VarHandle.getAcquire(Object...).
final boolean		

	getAndSet (boolean newValue)	Atomically sets the value to newValue and returns the old value, with memory effects as specified by VarHandle.getAndSet(Object...).
final boolean	**getOpaque**()	Returns the current value, with memory effects as specified by VarHandle.getOpaque(Object...).
final boolean	**getPlain**()	Returns the current value, with memory semantics of reading as if the variable was declared non-volatile.
final void	**lazySet** (boolean newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
final void	**set**(boolean newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setVolatile(Object...).
final void	**setOpaque** (boolean newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setOpaque(Object...).
final void	**setPlain** (boolean newValue)	Sets the value to newValue, with memory semantics of setting as if the variable was declared non-volatile and non-final.
final void	**setRelease** (boolean newValue)	Sets the value to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
String	**toString**()	

		Returns the String representation of the current value.
boolean	**weakCompareAndSet**(boolean expectedValue, boolean newValue)	**Deprecated.** This method has plain memory effects but the method name implies volatile memory effects (see methods such as compareAndExchange(boolean, boolean) and compareAndSet(boolean, boolean)).
final boolean	**weakCompareAndSetAcquire**(boolean expectedValue, boolean newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetAcquire(Objec
boolean	**weakCompareAndSetPlain**(boolean expectedValue, boolean newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetPlain(Object.
final boolean	**weakCompareAndSetRelease**(boolean expectedValue, boolean newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetRelease(Objec
final boolean	**weakCompareAndSetVolatil**(boolean expectedValue, boolean newValue)	Possibly atomically sets the value to newValue if the current value == expectedValue, with memory effects as

	specified by `VarHandle.weakCompareAndSet(Object...)`.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

AtomicBoolean

```
public AtomicBoolean(boolean initialValue)
```

Creates a new `AtomicBoolean` with the given initial value.

Parameters:

`initialValue` - the initial value

AtomicBoolean

```
public AtomicBoolean()
```

Creates a new `AtomicBoolean` with initial value `false`.

Method Details

get

```
public final boolean get()
```

Returns the current value, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Returns:

the current value

compareAndSet

```
public final boolean compareAndSet(boolean expectedValue,
                                   boolean newValue)
```

Atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndSet(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

weakCompareAndSet

```
@Deprecated(since="9")
public boolean weakCompareAndSet(boolean expectedValue,
                                 boolean newValue)
```

Deprecated.
This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(boolean, boolean)` and `compareAndSet(boolean, boolean)`). To avoid confusion over plain or volatile memory effects it is recommended that the method `weakCompareAndSetPlain(boolean, boolean)` be used instead.

Possibly atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

See Also:

`weakCompareAndSetPlain(boolean, boolean)`

weakCompareAndSetPlain

```
public boolean weakCompareAndSetPlain(boolean expectedValue,
                                      boolean newValue)
```

Possibly atomically sets the value to `newValue` if the current value `==` `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

set

```
public final void set(boolean newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setVolatile(Object...)`.

Parameters:

`newValue` - the new value

lazySet

```
public final void lazySet(boolean newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`newValue` - the new value

Since:

1.6

getAndSet

```
public final boolean getAndSet(boolean newValue)
```

Atomically sets the value to `newValue` and returns the old value, with memory effects as specified by `VarHandle.getAndSet(Object...)`.

Parameters:

`newValue` - the new value

Returns:

the previous value

toString

```
public String toString()
```

Returns the String representation of the current value.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current value

getPlain

```
public final boolean getPlain()
```

Returns the current value, with memory semantics of reading as if the variable was declared non-`volatile`.

Returns:

the value

Since:

9

setPlain

```
public final void setPlain(boolean newValue)
```

Sets the value to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.

Parameters:

`newValue` - the new value

Since:

9

getOpaque

```
public final boolean getOpaque()
```

Returns the current value, with memory effects as specified by `VarHandle.getOpaque(Object...)`.

Returns:

the value

Since:

9

setOpaque

```
public final void setOpaque(boolean newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setOpaque(Object...)`.

Parameters:

`newValue` - the new value

Since:

9

getAcquire

```
public final boolean getAcquire()
```

Returns the current value, with memory effects as specified by `VarHandle.getAcquire(Object...)`.

Returns:

the value

Since:

9

setRelease

```
public final void setRelease(boolean newValue)
```

Sets the value to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`newValue` - the new value

Since:

9

compareAndExchange

```
public final boolean compareAndExchange(boolean expectedValue,
                                        boolean newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchange(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeAcquire

```
public final boolean compareAndExchangeAcquire(boolean expectedValue,
                                               boolean newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeAcquire(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeRelease

```
public final boolean compareAndExchangeRelease(boolean expectedValue,
                                               boolean newValue)
```

Atomically sets the value to `newValue` if the current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeRelease(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

weakCompareAndSetVolatile

```
public final boolean weakCompareAndSetVolatile(boolean expectedValue,
                                               boolean newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSet(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetAcquire

```
public final boolean weakCompareAndSetAcquire(boolean expectedValue,
                                              boolean newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetAcquire(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetRelease

```
public final boolean weakCompareAndSetRelease(boolean expectedValue,
                                              boolean newValue)
```

Possibly atomically sets the value to `newValue` if the current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetRelease(Object...)`.

Parameters:

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.